UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2015/2016

SEPTEMBER EXAMINATION

UECS3253 WIRELESS APPLICATION DEVELOPMENT

FRIDAY, 18 SEPTEMBER 2015 TIME : 2.30 PM – 4.30 PM (2 HOURS)

BACHELOR OF SCIENCE (HONS) SOFTWARE ENGINEERING

Instructions to Candidates:

This question paper consists of FIVE (5) questions.

Answer ANY FOUR (4) questions only.

Each question carries 25 marks.

Note: Should a candidate answer more than FOUR (4) questions, marks will only be awarded for the FIRST FOUR (4) questions in the order the candidate submits the answer.

Answer the questions in the answer book provided.

UECS3253 WIRELESS APPLICATION DEVELOPMENT

Q1. (a) (i) Write a short review to compare and contrast the *Android* platform by Google and the *iOS* platform by Apple. (8 marks)

(ii) What advantages do *mobile hybrid apps* have over *mobile native apps* and *mobile web apps*? (4 marks)

(b) Answer the following questions based on the content of the **AndroidManifest.xml** manifest file of an Android app in Figure 1.1 in Page 3.

(i) List the *services* of this Android app. (2 marks)

(ii) List the *widgets* of this Android app. (2 marks)

(iii) What is the *hierarchical parent* of the activity **DeltaActivity**? (2 marks)

(iv) A new activity **OmegaActivity** is added to the project. Write the XML code segment for this new activity that will be included in the above manifest file. This activity has **DeltaActivity** as its hierarchical parent. (7 marks)

[Total : 25 marks]

UECS3253 WIRELESS APPLICATION DEVELOPMENT

Q1(b). (Continued)

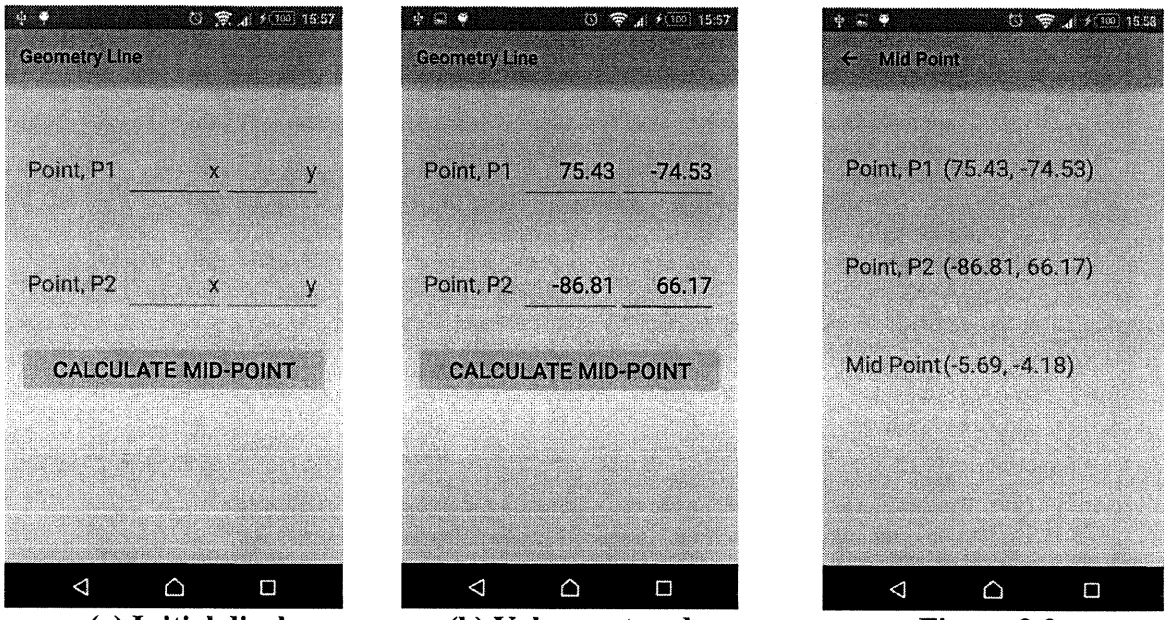
```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.jamesooi.asimpleapp" >
    <application android:allowBackup="true" android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name" android:theme="@style/AppTheme" >
        <activity android:name=".HomeActivity" android:label="@string/app_name" >
            <intent-filter><action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" /></intent-filter>
        </activity>
        <activity
            android:name=".AlphaActivity" android:label="@string/title_activity_alpha"
            android:parentActivityName=".HomeActivity" >
            <meta-data android:name="android.support.PARENT_ACTIVITY"
                android:value="com.jamesooi.asimpleapp.HomeActivity" />
        </activity>
        <activity
            android:name=".BetaActivity" android:label="@string/title_activity_beta"
            android:parentActivityName=".AlphaActivity" >
            <meta-data android:name="android.support.PARENT_ACTIVITY"
                android:value="com.jamesooi.asimpleapp.AlphaActivity" />
        </activity>
        <activity
            android:name=".GammaActivity" android:label="@string/title_activity_gamma"
            android:parentActivityName=".HomeActivity" >
            <meta-data android:name="android.support.PARENT_ACTIVITY"
                android:value="com.jamesooi.asimpleapp.HomeActivity" />
        </activity>
        <activity
            android:name=".DeltaActivity" android:label="@string/title_activity_delta"
            android:parentActivityName=".AlphaActivity" >
            <meta-data android:name="android.support.PARENT_ACTIVITY"
                android:value="com.jamesooi.asimpleapp.AlphaActivity" />
        </activity>
        <service
            android:name=".CatService" android:enabled="true" android:exported="true" >
        </service>
        <service
            android:name=".DuckService" android:enabled="true" android:exported="true" >
        </service>
        <receiver android:name=".SpringWidget" >
            <intent-filter>
                <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
            </intent-filter>
            <meta-data android:name="android.appwidget.provider"
                android:resource="@xml/spring_widget_info" />
        </receiver>
        <receiver android:name=".AutumnWidget" >
            <intent-filter>
                <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
            </intent-filter>
            <meta-data android:name="android.appwidget.provider"
                android:resource="@xml/autumn_widget_info" />
        </receiver>
    </application>
</manifest>
```

Figure 1.1

UECS3253 WIRELESS APPLICATION DEVELOPMENT

Q2. (a) Explain the purpose of using *implicit intents* in Android apps. (4 marks)

(b) An Android app to calculate the mid-point coordinates of a line defined by two points is as shown in screenshots of the activities **MainActivity** and **DisplayMidPointActivity** in Figure 2.1 and 2.2 respectively.



(a) Initial display **(b) Values entered**
Figure 2.1: MainActivity

Figure 2.2: DisplayMidPointActivity

You are provided with two Java classes – **Point** and **Line**. These classes are as illustrated in the class diagram in Figures 2.3. The names of the members of the classes are self-explanatory.

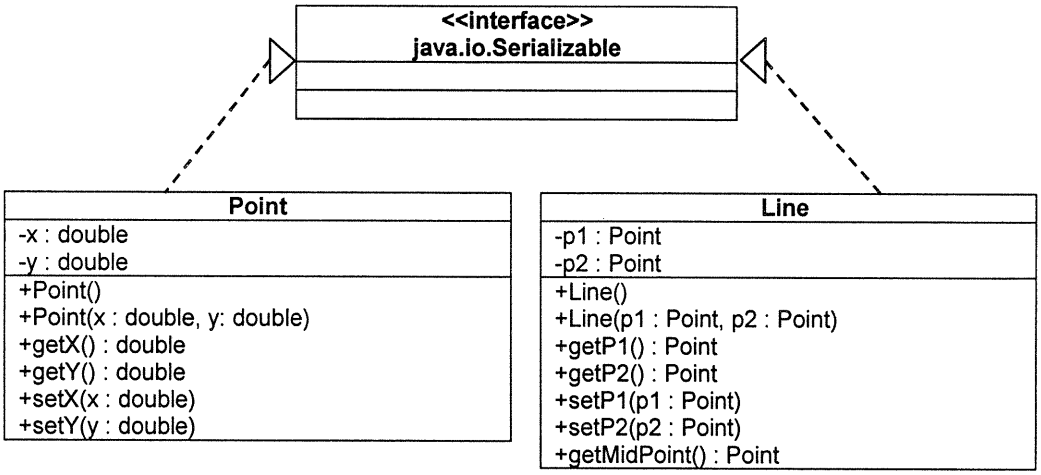


Figure 2.3: Point and Line classes

UECS3253 WIRELESS APPLICATION DEVELOPMENT

Q2(b). (Continued)

(i) Write the XML code segment to define all the strings required in this app in the **strings.xml** resource file. (2 marks)

(ii) Figure 2.4 shows the **activity_main.xml** layout file which will be used in the activity **MainActivity**.

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical">
    <LinearLayout
        android:layout_width="fill_parent" android:layout_height="wrap_content"
        android:orientation="horizontal">
              <!-- CODE OMITTED -->
    </LinearLayout>
    <LinearLayout
        android:layout_width="fill_parent" android:layout_height="wrap_content"
        android:orientation="horizontal">
              <!-- CODE OMITTED -->
    </LinearLayout>
    <!-- CODE OMITTED -->
</LinearLayout>
```

A B C

Figure 2.4

Complete the code in the sections marked **A** and **C** only based on the screenshots in Figures 2.1 and the following requirements:

- The **EditText** views are assigned the **id** *point_one_x, point_one_y, point_two_x* and *point_two_y* respectively.
- The **EditText** views accept only single line, signed floating-point numbers as input.
- The button calls the method **calculateMidPoint** when clicked.

(8 marks)

(iii) Write the required Java code for the method **calculateMidPoint** which is invoked when the button in the activity **MainActivity** is clicked. The method sends two **Point** objects as a bundle of extras to the activity **DisplayMidPointActivity** and starts it. (7 marks)

(iv) Write the Java code segment for the **onCreate** callback method in the activity **DisplayMidPointActivity** to receive the intent from **MainActivity** and display the output as shown in Figure 2.2. Assume that three **TextView** views with the **id** *point_one, point_two* and *mid_point* are used to display the coordinates of P1, P2 and Mid Point respectively in the format **(-xx.xx, -yy.yy)** (4 marks)

[Total : 25 marks]

UECS3253 WIRELESS APPLICATION DEVELOPMENT

Q3. (a) (i) A member of an Android app development team is tasked to create an activity to display a user registration form. Explain whether he should create the user interface *programmatically* or using *XML layout file*. (2 marks)

(ii) Write Java code segment to create an **EditText** view programmatically and set the text size to 20 and set the hint to the string ***Year***. (3 marks)

(b) A simple picture gallery app with two activities is as shown in Figures 3.1 and 3.2. In Figure 3.1, the **GalleryActivity** uses a **GridView** to display a grid of thumbnail images obtained from drawable resources. Clicking on the thumbnail opens the **PictureActivity** as shown in Figure 3.2 to display the corresponding full-sized image obtained from drawable resources and a description.

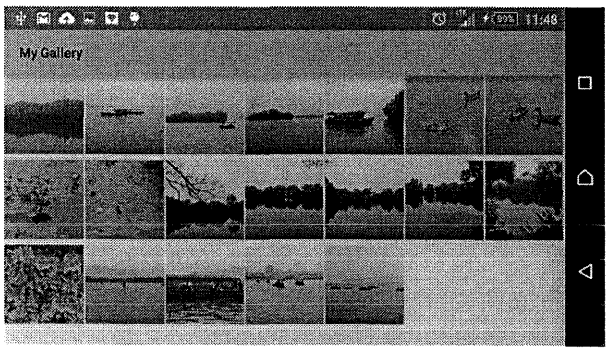


Figure 3.1: GalleryActivity

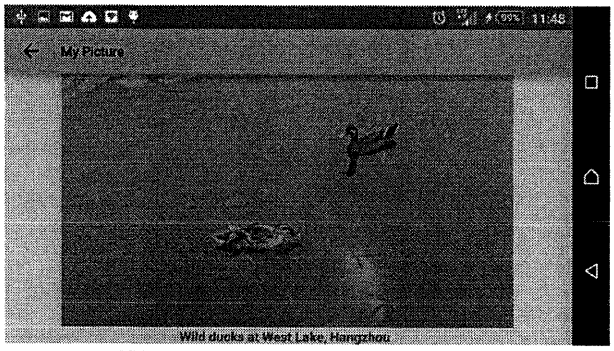


Figure 3.2: PictureActivity

Figure 3.3 shows the class diagrams for the classes **Gallery** and **Picture**. Both classes implement the **Serializable** interface, but are NOT denoted in the diagrams. The names of the members of both classes are self-explanatory. The app also uses a custom class named **ImageAdapter** which extends **BaseAdapter**.

Gallery
-pictures : ArrayList<Picture> -currentPicture : int
+Gallery() +getPicture(index : int) : Picture +getNextPicture() : Picture +getPreviousPicture() : Picture +getSize() : int +addPicture(picture : Picture)

Picture
- pictureResource: int -thumbnailResource : int -description : String
+Picture() +Picture(pictureResource : int, thumbnailResource: int, description : String) +getPictureResource() : int + getThumbnailResource() : int +getDescription(x : double) +setPictureResource(pictureResource : int) +setThumbnailResource(thumbnailResource : int) +setDescription(description : String)

Figure 3.3: Gallery and Picture classes

(i) Write the XML code segment to define the TWO (2) views of **PictureActivity.** Assume that the views are to be placed within a **LinearLayout** view group with the orientation set to vertical. (3 marks)

Q3(b). (Continued)

(ii) Figure 3.4 shows the partial code segment for the **GalleryActivity** class.

```
// package & import statements intentionally omitted

public class GalleryActivity extends Activity {
      // CODE OMITTED: Data fields required
                                                        A
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_gallery);

        final Gallery gallery = new Gallery();
        gallery.addPicture(new Picture(R.drawable.wl_01, R.drawable.tn_wl_01,
          "West Lake, Hangzhou"));
        // ...
        // more addPicture() method invocations intentionally omitted...

        // CODE OMITTED: Set the adapter to ImageAdapter and the OnItemClickListener
                                                        B
    }
}
```

Figure 3.4

Complete the code in the sections marked **A** and **B** based on the following requirements:

[A]
- Define any data fields required in **GalleryActivity** class

[B]
- Obtain a reference to the **GridView** view with the **id** ***grid_view***.
- Set the adapter of the **GridView** view above with an instance of **ImageAdapter** custom class to display the thumbnails from the **Gallery** instance referenced by the variable **gallery**.
- Implement the **OnItemClickListener** to start the activity **PictureActivity** to display the corresponding full-sized image of the thumbnail clicked. (8 marks)

(iii) Write Java code to implement the method **getView()** of the class **ImageAdapter**. The method returns an **ImageView** object that has been set to display a single thumbnail image. Assume that the **gallery** data field in **ImageAdapter** contains reference to a valid **Gallery** object. (4 marks)

(iv) Write the Java code segment for the **PictureActivity** to receive the intent from **GalleryActivity** and sets the views you defined in part (i) to display the full-sized image and description. (5 marks)

[Total : 25 marks]

UECS3253 WIRELESS APPLICATION DEVELOPMENT

Q4. (a) Explain the three (3) uses of *data persistence* in Android apps. (6 marks)

(b) Figure 4.1 shows an activity of an Android app that accepts user input about a person. Clicking on the *save* button will save the user input in an SQLite database. Figure 4.2 shows the class diagrams of the classes **Person** and **DBHelper** that is used in this app.

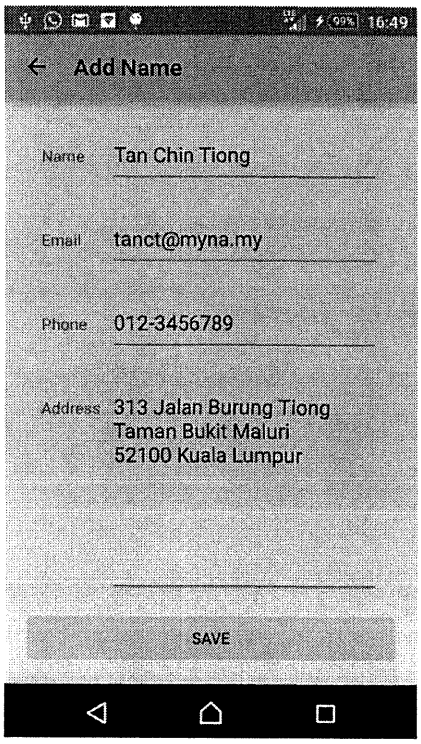


Figure 4.1: Activity

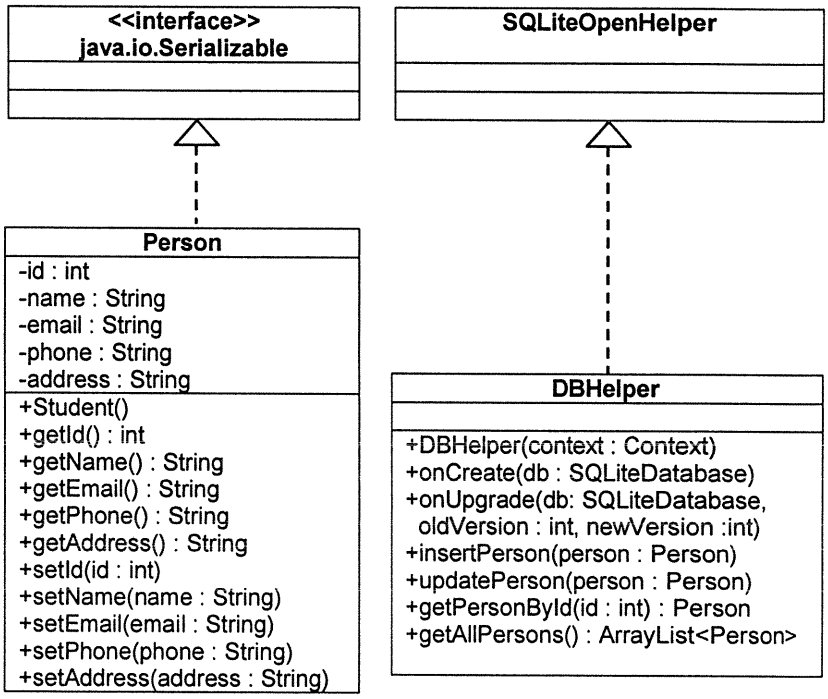


Figure 4.2: Class diagrams of Person and DBHelper

(i) Write the Java code for the **onCreate** method of **DBHelper** class to create the table **person.** (4 marks)

(ii) Write the Java code for the **insertPerson** method of **DBHelper** class to insert a record to the table **person.** (6 marks)

(iii) Write the Java code for the **getAllPersons** method of **DBHelper** class to get all records of the table **person.** (9 marks)

[Total : 25 marks]

UECS3253 WIRELESS APPLICATION DEVELOPMENT

Q5. (a) In addition to using files, shared preferences and SQLite, Android apps can also implement data persistence by storing data on servers connected via the Internet. Two possible methods to implement this is via *HTTP* and *network sockets*. Give a scenario and explain when one would consider using *network sockets* instead of *HTTP*. (6 marks)

(b) Network operations in Android apps are usually performed on a separate thread. Explain the reason for doing so. (6 marks)

(c) An Android app connects to an HTTP web service to read an image file to be displayed in an **ImageView** view.

(i) Write the XML code segment for the **AndroidManifest.xml** file to set the required permissions for connecting to the network. (4 marks)

(ii) Write the Java code segment to connect to the URL ***https://cdn.belatuk.net/images/2015/02/19/001.jpg*** to download the image and convert the input stream into a **Bitmap** object. (9 marks)

[Total : 25 marks]